

Reviewer Pack — Minimal p-Adic Logarithmic Order and Communication Complexit...

Entry c94f3b3866d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 21:12:39 UTC

Minimal p-Adic Logarithmic Order and Communication Complexity Rank

Entry ID: c94f3b3866d3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 21:12:39 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (Logarithmic Order) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any CNF φ with m clauses and n variables, the minimal logarithmic order of the p-adic expansion of the number of satisfying assignments (φ) is lower-bounded by the communication complexity rank ($\text{rank}(\varphi)$) to within a multiplicative factor of $O(p^{(n/2)})$.

Rationale (proposer's reasoning):

p-adic logarithmic order captures the 'difficulty' of computing an invariant that uniquely identifies a mathematical object, and it has been used in other areas to describe properties of algebraic objects. By linking this to communication complexity rank, we might expose a new avenue for proving lower bounds on communication complexity, which is a well-studied area with many open problems.

Taxonomy category: p-adic_analysis (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4f2782eb95c11556

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the p-adic logarithmic order of satisfying assignments and the communication complexity rank exceeds 0.8 for at least 80% of the seeds, with no seed producing a correlation coefficient below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=120.4s

5.1 Generated Python source

```
import random
import math

def generate_cnf(m, n):
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n) * (-1 if random.choice([True, False]) else 1) for _ in range(n)]
        cnf.append(clause)
    return cnf

def count_satisfying_assignments(cnf):
    n = len(cnf[0])
    assignments = [i for i in range(-2**n + 1, 2**n)]
    valid_assignments = []
    for assignment in assignments:
        if all((lit > 0 and assignment & lit) or (lit < 0 and not assignment & -lit) for clause, lit in zip(cnf, assignment)):
            valid_assignments.append(assignment)
    return len(valid_assignments)

def p_adic_log_order(x):
    if x == 0:
        return float('-inf')
    order = 0
    while x % 2 == 0:
        x //= 2
        order += 1
    return order

def communication_complexity_rank(cnf):
    m, n = len(cnf), len(cnf[0])
    rank = 0
    for i in range(m):
        rank = max(rank, sum(1 for lit in cnf[i] if abs(lit) <= n))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

m_values = [5, 10, 15, 20, 30, 40]
results = []

for m in m_values:
    cnf = generate_cnf(m, m)
    num_satisfying_assignments = count_satisfying_assignments(cnf)
    p_adic_order_value = p_adic_log_order(num_satisfying_assignments)
    rank_value = communication_complexity_rank(cnf)

    results.append({
        "m": m,
        "num_satisfying_assignments": num_satisfying_assignments,
        "p_adic_order_value": p_adic_order_value,
        "rank_value": rank_value
    })

if not results:
    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

p_adic_order_values = [r["p_adic_order_value"] for r in results]
rank_values = [r["rank_value"] for r in results]

mean_p_adic_order = sum(p_adic_order_values) / len(p_adic_order_values)
std_p_adic_order = math.sqrt(sum((x - mean_p_adic_order) ** 2 for x in p_adic_order_values) / len(p_adic_order_values))
mean_rank = sum(rank_values) / len(rank_values)

correlation_coefficient = (sum((p_adic_order_values[i] - mean_p_adic_order) * (rank_values[i] - mean_rank) for i in range(len(results))) / (len(results) * std_p_adic_order * math.sqrt(sum((x - mean_rank) ** 2 for x in rank_values) / len(rank_values))))

return {
    "metric_name": "communication_complexity_rank",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(m_values),
    "conjecture_holds": correlation_coefficient > 0.8,
    "counterexample": "" if correlation_coefficient >= 0.5 else f"correlation_coefficient={correlation_coefficient}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```
mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) /
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value:.2f} std={std_metric_value:.2f} support_
elif any(r["counterexample"] != "" for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["countere
    print(f"RESULT: FALSIFIED counterexample=\"{next(result['counterexample'] for result in re
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction:.2f}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': 0.38655795682095406, 'instan
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test result indicates that the conjecture does not hold for at least one seed, as the correlation coefficient between the p-adic logarithmic order | next: Investigate further to identify why the correlation coefficient is low and whether there are specific types of CNFs that lead to such a discrepancy.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23438
2	preregistration	ollama_remote	glm4:latest	0	0	14249
3	novelty	ollama_remote	glm4:latest	0	0	15305
4	novelty	ollama_remote	glm4:latest	0	0	11234
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16227
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22249
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18681
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18647
9	judge	ollama_remote	glm4:latest	0	0	44040

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 184071 ms total latency. Provider mix: {'ollama remote': 9}

(full prompt+response transcripts available in research/audit/c94f3b3866d3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c94f3b3866d3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c94f3b3866d3.tar.gz (if generated)